

On the Hellgate Cluster, we utilize SCHEDMD's Slurm, a work load manager. Slurm is a job scheduling system for Linux clusters, which allocates resources to users, manages job execution and handles job queues.

More information can be found at [Slurm's Documentation](#).

SLURM Commands

Sbatch

Used to submit a job script for later execution. The script will typically contain one or more commands.

We recommend using sbatch, it allows users to more easily specify resource allocation, schedule job times, create error logs and keep the Slurm job running regardless of the user closing the terminal.

Here is an example of what a sbatch script would look like: (We would create a file which ends with .sh)

```
#!/bin/bash
#SBATCH --job-name=cpu_job
##SBATCH --mail-type=END,FAIL # Mail events (NONE, BEGIN, END, FAIL, ALL)
##SBATCH --mail-user= # Where to send mail
#SBATCH --partition='cpu(all)'
#SBATCH --nodes=1 # Number of nodes
#SBATCH --ntasks=20 # Number of simultaneous processes
#SBATCH --cpus-per-task=1 # Number of cores per process
#SBATCH --mem=100G # Memory (RAM)
#SBATCH --time=0-8
#SBATCH --output=out.txt
#SBATCH --error=err.txt

### COMMAND TO RUN ###
echo "This is a CPU job"
sleep 30
```

To run the script, we would reference the file after calling the sbatch command.

```
sbatch <SCRIPT-NAME>.sh
```

Sbatch has the ability to request multiple jobs using arrays, allowing us to run one script but execute multiple processes. Resource specification will be for each job. This can be done with a script similar to:

```
#!/bin/bash
#SBATCH --job-name=array_job_%a # %A is job id, %a is array id
##SBATCH --mail-type=END,FAIL # Mail events (NONE, BEGIN, END, FAIL, ALL)
##SBATCH --mail-user= # Where to send mail
#SBATCH --partition='cpu(all)'
#SBATCH --nodes=1 #Number of nodes
#SBATCH --array=1-8
#SBATCH --ntasks=4 # Number of simultaneous processes per job
#SBATCH --cpus-per-task=1 # Number of cores per process
#SBATCH --mem=32G # Memory (RAM) per job
#SBATCH --time=0-8
#SBATCH --output=./out_%A/out_job%a.txt
#SBATCH --error=./err_%A/err_job%a.txt

### COMMAND TO RUN ###
echo "This is array job $SLURM_ARRAY_TASK_ID"
sleep 30
```

Srun

Used to submit a job for execution or initiate job steps in real time. srun has a wide variety of options to specify resource requirements. A job can contain multiple job steps executing sequentially or in parallel on independent or shared resources within the job's node allocation.

Using srun without salloc, will both allocate resources and execute the tasks as a single job and job step

```
srun --job-name=myjob --time=00:05:00 --nodes=2 --ntasks-per-node=4 --cpus-per-task=1 <COMMAND>
```

This will update the terminal to be inside the created slurm instance, allowing for executing code within the slurm instance.

Exiting the session will terminate the slurm instance, to work around this issue, we recommend using tmux, a Linux shell which allows users to create psuedo-terminal sessions, to keep the slurm instance alive, while letting the user to still use the same terminal.

Squeue

Reports the state of jobs or job steps. It has a wide variety of filtering, sorting, and formatting options. By default, it reports the running jobs in priority order and then the pending jobs in priority order.

```
squeue
```

Using watch will keep a persistent window which updates every 2s

```
watch squeue
```

Squeue can be filtered to only your job using:

```
squeue --me
```

Scancel

Used to cancel a pending or running job or job step. It can also be used to send an arbitrary signal to all processes associated with a running job or job step.

```
scancel <JOB_ID>
```

Sacct

Used to report job or job step accounting information about active or completed jobs.

```
sacct --jobs=<JOB_ID> --  
format=JobID,JobName,Elapsed,systemcpu,totalcpu,avecpu,cputime,maxvmsize,maxr  
ss
```

Sinfo

Used to report the state of partitions and nodes managed by Slurm. It has a wide variety of filtering, sorting, and formatting options.

```
sinfo
```

SSHing to Nodes

It is possible to SSH into a node that is currently running one of our jobs. This can provide us with useful information on what is going on inside the node.

In this example, we have a job running on hgcpu1-1 and would like to see what processes are running. SLURM is configured to allow us to easily SSH in, this can be done simply with:

```
ssh hgcpu1-1
```

We will then use ‘top’ to provide us with resource usage on the node:

```
top
```